Robotic Car Painting Cell Simulation

Technical report: Yuktii AI Labs Robotics Simulation Intern assessment

Vinit | Mechanical Engineering (Mechatronics and Robotics), VIT Bhopal | October 2026

1. Overview

For this assessment I built a small robotic painting cell in RoboDK. One ABB IRB 4600 robot stands on a pedestal next to a Tesla Model 3 mesh that I scaled to the 4,800 mm reference length, and it carries a simple spray gun model. I wrote six programs that paint zigzag strokes over the roof, the hood and the trunk, each one split into a near half and a far half. RoboDK's own program check reports every move in all six programs as reachable and collision-free. I also tried the side panels and did not get them working; section 8 explains where that stopped.

2. Software used

- **RoboDK** (Windows, free license) for the station, the robot library, the programs and the collision checks.
- **Python 3.13** with the RoboDK API (robodk 6.0.2), numpy and trimesh. Almost everything in the station was made by script, so the project can be rebuilt from the scripts in the GitHub repository.
- **Car model:** a free Tesla Model 3 mesh in GLB format, converted to STL.

3. Robot selected

ABB IRB 4600-20/2.50, a six-axis industrial arm with a 2.5 m reach, taken from the RoboDK library. I picked it because it was available and the reach is long enough to get over the roof of a car from one side. A real paint line would use a dedicated paint robot, but the approach in this project (strokes that follow the body, collision checking, repositioning the car) does not depend on which arm is used.

4. Cell layout

Item	Size and position
Floor	10,000 × 6,000 mm, 100 mm thick, top surface at Z = 0
Side walls	Two walls, 3,500 mm high, along the long sides (Y = ±3,050 mm). The two ends are open so a car can enter and leave, like the arrows in the reference layout
Pedestal	700 × 700 × 500 mm block at Y = −1,900 mm
Robot	IRB 4600-20/2.50 standing on the pedestal, base turned 90° about Z so it faces the car
Car	Tesla Model 3 mesh, 4,800 × 2,235 × 1,440 mm, placed on a reference frame called Car_Frame
Paint gun	A cylinder with a cone on the end, fixed to the robot flange. The tool centre point is at the nozzle tip, 300 mm from the flange

The brief gives about 4,800 × 1,900 × 1,600 mm. I scaled the mesh by its length only, so the proportions of the model stay as they are. The 2,235 mm width includes the mirrors (the body itself is roughly 1,980 mm wide) and the height came out at 1,440 mm instead of 1,600 mm. I took that as close enough for approximately.

Figure 1 - INSERT SCREENSHOT: top view of the whole cell (robot, pedestal, car, walls).

5. Approach

The reach of the robot decided most of the design. The car is 4.8 m long and the arm reaches 2.5 m, so from one spot it cannot paint the whole car. I could have put the robot on a linear track. I moved the car instead, by changing the position and rotation of Car_Frame, because that was simpler to set up in RoboDK and it is close to what a conveyor with a turntable would do in a real cell.

The steps were:

1. **Car.** The GLB file has Y pointing up, while RoboDK uses Z. A script rotates the mesh 90° about X, turns it so the length runs along X, scales it to 4,800 mm, centres it and puts the wheels on Z = 0, then saves an STL.
2. **Cell.** The floor, walls and pedestal are boxes made with trimesh and loaded into RoboDK through the API.
3. **Tool.** The gun is a cylinder with a cone on the end. The tool centre point sits at the tip.
4. **Paths.** A script reads the car mesh and builds the painting strokes (section 6).
5. **Positioning.** Every program has its own Car_Frame position and rotation, listed in the table in section 8.
6. **Checking.** RoboDK's program check reports reachability and collisions together.

6. Trajectory and path planning

Every program uses the same raster pattern. The robot paints straight strokes in one direction and steps sideways after each one, and the travel direction flips on every stroke, so it goes back and forth instead of returning to the start each time.

Parameter	Value
Stand-off (nozzle to surface)	200 mm
Distance between strokes	120 mm
Distance between points on a stroke	100 mm
Approach and retreat height	80 mm above the first and last point of a stroke
Tool direction	Pointing straight down
Move types	Joint move to the approach point, linear moves along the stroke

To follow the body shape, the script takes each (x, y) point on a stroke, finds the highest mesh vertex inside an 80 mm square window around it, and adds the 200 mm stand-off. The path then rises and falls with the roof line, the hood and the trunk. It does not use real surface normals.

The 120 mm spacing is meant to give about 50% overlap between neighbouring strokes if the spray fan is around 240 mm wide at that distance. That width is my own assumption. I did not model a real spray pattern.

The surfaces are defined like this:

- **Roof:** mesh vertices above 93% of the maximum height, with 100 to 150 mm trimmed off the edges.
- **Hood and trunk:** the two ends of the car, X from 1,500 to 2,200 mm and X from −2,200 to −1,500 mm in car coordinates.
- Each surface is split at the car centreline. The near half is painted with the car in one position, and the far half with the car turned 180°.

The hood and trunk programs also start and end with a move to a Home joint position.

Figure 2 - INSERT SCREENSHOT: the Paint_Roof path with the gun above the roof.

Figure 3 - INSERT SCREENSHOT: one of the hood or trunk programs running, with the car moved along the cell.

7. Collision checking

I used the RoboDK collision map. The robot links, the gun, the car, the floor, the walls and the pedestal are all checked against each other. I switched off only the pairs that touch by design: the robot base against the pedestal, and the gun against the last wrist joints, since the gun is bolted to the flange. Each program was then run through RoboDK's program check.

To make sure the check really sees the gun, I put the robot at a roof target (R010 in Paint_Roof) and lifted only the car model:

Car lifted by	Collisions reported
0 mm	none
100 mm	none
250 mm	gun and car
600 mm	three robot links and the car

Problems the checks showed

- **Unreachable roof stroke.** My first attempt had the robot on the floor at Y = −2,200 mm with a 250 mm stand-off, and the second stroke (target R019) could not be reached. I raised the robot on a 500 mm pedestal, moved the base to Y = −1,900 mm, cut the

stand-off to 200 mm and trimmed the stroke ends. After that every move was reachable.

- **False clean results.** My first runs said there were no collisions, but the gun was not really being checked. Two things were wrong. The gun pairs were switched off in the collision map, and once I switched them on, every target collided because the gun model sat 300 mm too low (its geometry started at the nozzle tip and ran 300 mm past it). I moved the model 300 mm back along the tool axis and the collisions at the normal car position went away.
- **Far halves of the hood and trunk.** After the 180° turn of the car these programs failed at instruction 18 as unreachable. Turning the tool orientation of those targets by 180° about Z, so the gun points the same way in the cell as in the near programs, fixed it.
- **Hood near program.** It reported a collision at its first move because the robot was left in the pose of the previous program. Starting every hood and trunk program from a Home joint position solved it.

Figure 4 - INSERT SCREENSHOT: the Collision map window.

Figure 5 - INSERT SCREENSHOT: terminal output of the lift test and a clean check result.

8. Results

Each program was run through the RoboDK program check after the fixes above. The time is the simulated program time reported by RoboDK.

Program	Car_Frame X (mm) and turn	Targets	Instructions	Time (s)	Valid ratio	Collisions
Paint_Roof	0, no turn	85	87	43.6	1.0	none
Paint_Roof_Far	977, 180°	68	70	44.3	1.0	none
Paint_Hood_Near	−1,850, no turn	80	84	43.9	1.0	none
Paint_Hood_Far	1,850, 180°	70	74	38.3	1.0	none
Paint_Trunk_Near	1,850, no turn	80	84	42.9	1.0	none
Paint_Trunk_Far	−1,850, 180°	70	74	38.0	1.0	none

All six programs come back with a valid ratio of 1.0 and no collisions, and together they cover the roof, the hood and the trunk. The two halves of each surface meet at the car centreline, where the overlap between strokes is smaller than elsewhere. The outer 100 to 150 mm around the edges of each region is left unpainted.

Figure 6 - INSERT SCREENSHOT: terminal output showing the check results for the six programs.

Side panels (not completed)

For the sides the gun has to point sideways at the car. With the car in its normal place, the move to target 10 of the first stroke was unreachable. I moved the car 500 mm away from the robot, and the program then failed with a collision at its first approach move. A diagnostic run showed that 18 of the 115 targets, the strokes at 450 and 550 mm height around the middle of the door (X about −400 to +400 mm), had robot links hitting the pedestal, and that the swing from Home to the first target passed through the car. I wrote a new version that starts the strokes at 650 mm and adds a via-point, but RoboDK's free license then started refusing API calls, so I could not run it. The side panels are therefore not part of the results.

9. Limitations and next steps

- The tool points straight down on every surface. On the sloped hood and trunk it is not normal to the surface. Estimating normals from the mesh and tilting the tool would be the first improvement.
- Heights come from a vertex window and not from real surface normals, so small details are smoothed out.
- The stand-off and the stroke spacing are my own choices. There is no spray model, no paint thickness or coverage simulation, and the car does not change colour in the simulation.
- The side and lower body panels are not covered. A lower mount for the arm, a tilted tool or a linear track are the options I would try.
- The car is moved by editing Car_Frame. A real cell would need a conveyor or turntable with its own motion and timing.
- The free RoboDK license limits how many times the station can be saved, and the API calls were blocked at the end. For that reason the scripts are the main record of the project.

10. Files

The simulation files, screenshots and demo video are in the Google Drive folder. The scripts and the README are in the GitHub repository. Both links are in the submission email.